

ÉPISODE 1

Les fichiers en Python

Parce qu'un programme qui oublie tout à la fin, c'est un poisson rouge.

Licence Informatique 2e année · Algo & Prog LS3 · Ph. Huneil · 2026-2027

5

5

Pourquoi des fichiers ?

- **Les variables vivent en mémoire vive : à la fin du programme, tout disparaît.**
- Un fichier est un emplacement sur le disque où les données survivent à la fin du programme (et à l'extinction de la machine).
- On y stocke tout : texte, nombres, images, sons, vidéos, programmes...
- **Deux grandes familles :**
 - Fichiers texte : du texte brut (code Python, données, notes). Lisibles par un humain.
 - Fichiers binaires : images, vidéos, exécutables. Lisibles par un programme.

 **Exemple concret**

Une appli qui relève la hauteur des vagues au Diamant chaque heure : sans fichier, elle recommence de zéro à chaque lancement. Avec un fichier, elle a un historique.

 **Question flash**

Un fichier .py est-il texte ou binaire ? Et un .pptx ? Et un .csv ? (Indice : ouvrez-les avec un éditeur de texte.)

Licence Informatique 2e année · Algo & Prog LS3 · Ph. Huneil · 2026-2027

6

6

Où sont mes fichiers ? La bibliothèque os

- Le module os fait dialoguer Python avec le système d'exploitation : répertoires, renommage, listing...
- Il ne copie pas de fichiers (ça, c'est le rôle de shutil).

Fonction	Ce qu'elle fait
os.getcwd()	Répertoire courant (« où suis-je ? »)
os.chdir(rep)	Se déplacer dans le répertoire rep
os.listdir(rep)	Liste des fichiers et dossiers de rep
os.mkdir(rep)	Créer un répertoire
os.rename(src, dest)	Renommer (ou déplacer)
os.remove(f) / os.rmdir(rep)	Supprimer un fichier / un répertoire vide
os.system(cmd)	Exécuter une commande du système (à manier avec prudence)

Essayez

```
import os
print(os.getcwd())
# macOS : /Users/philippe
# Windows : C:\Users\phili
os.mkdir("releves")
os.chdir("releves")
print(os.listdir(".")) # []
```

Piège classique

os.remove ne demande pas de confirmation. Il n'y a pas de corbeille.

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Huneil - 2026-2027

7

7

Chemins : os.path, votre GPS de fichiers

Fonction	Rôle
os.path.join(a, b, ...)	Assemble un chemin avec le bon séparateur (/ ou \)
os.path.split(chemin)	Renvoie (répertoire, nom_de_fichier)
os.path.exists(chemin)	Vrai si le chemin existe
os.path.isfile(chemin)	Vrai si c'est un fichier
os.path.isdir(chemin)	Vrai si c'est un répertoire
os.walk(chemin)	Parcourt récursivement toute l'arborescence

Astuce moderne : le module pathlib (Path) fait la même chose avec une syntaxe objet — à explorer avec l'IA.

Avec l'IA

« Réécris cet exemple avec pathlib et expliquez ligne par ligne la différence. » Puis testez les deux versions.

```
import os
chemin = os.path.join(os.getcwd(), "bidon.py")
print(chemin)
# /Users/philippe/bidon.py (macOS)
# C:\Users\phili\bidon.py (Windows)
rep, fic = os.path.split(chemin)
print("Le dossier :", rep)
print("Le fichier :", fic)
if not os.path.exists(chemin):
    print("Ce fichier n'existe pas (encore).")
```

Pourquoi join et pas + ?

"data" + "/" + "f.txt" casse sous Windows. os.path.join choisit le séparateur du système : votre code tourne partout.

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Huneil - 2026-2027

8

8

Copier : la bibliothèque shutil

- **shutil.copyfile(src, dest)**
 - copie le contenu de src dans le fichier dest (chemins absolus ou relatifs) ;
 - dest doit être un nom de fichier ; les droits ne sont pas copiés.
- **shutil.copy(src, dest)**
 - comme copyfile, mais dest peut être un dossier ;
 - copie aussi les droits d'accès.
- *Bonus : shutil.copytree (un dossier entier), shutil.move (déplacer).*

```
import shutil, os
os.mkdir("sauvegarde")
shutil.copyfile("notes.txt", "notes_v2.txt")
shutil.copy("notes.txt", "sauvegarde")
# -> sauvegarde/notes.txt
```



Défi : le sauvegardeur

Écrire un script qui copie tous les fichiers .py du répertoire courant dans un dossier « backup » (à créer s'il n'existe pas). Bonus : ajouter la date dans le nom du dossier.

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Hunel - 2026-2027

9

9

PARTIE 2

Lire et écrire dans un fichier



Ouvrir, travailler, refermer. Comme un frigo.

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Hunel - 2026-2027

10

10

Le fichier logique : open() et close()

- Pour manipuler le contenu d'un fichier, on crée un objet fichier (ou « fichier logique ») avec open().
- open(nom, mode) renvoie cet objet ; on travaille dessus ; puis on le referme avec close().
- Tant que le fichier est ouvert, le système lui réserve des ressources et les écritures peuvent rester en attente : fermer, c'est garantir que tout est bien sur le disque.**

```
ficLog = open("test.txt", "r") # ouvre
# ... on travaille ...
ficLog.close()                # ferme

# Variante moderne et sûre : with
with open("test.txt", "r") as f:
    contenu = f.read()
# ici f est déjà fermé, même en cas d'erreur
```

Mode	Signification
"r"	Lecture seule (erreur si le fichier n'existe pas)
"w"	Écriture : crée le fichier, ou l'écrit s'il existe !
"a"	Ajout : écrit à la fin, sans rien effacer
"rb" / "wb"	Idem en binaire
"r+"	Lecture et écriture



Le mode "w" ne prévient pas

open("rapport.txt", "w") efface instantanément le rapport existant.
Avant d'écrire, demandez-vous : est-ce que je veux "w" ou "a" ?

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Huneil - 2026-2027

11

11

Lire : quatre façons, un seul curseur

Soit f = open("entree.txt", "r"). Le fichier possède une position courante (un curseur) qui avance à chaque lecture.

Méthode	Renvoie
f.read()	Tout le fichier, en une seule chaîne
f.read(n)	Les n prochains caractères
f.readline()	La prochaine ligne (avec son \n) ; "" à la fin du fichier
f.readlines()	Toutes les lignes restantes, dans une liste
for ligne in f:	Parcours ligne à ligne (le plus « pythonique »)

```
with open("plages.txt") as f:
    for ligne in f:
        nom = ligne.strip() # enlève le \n
        print(nom.upper())
```

```
# plages.txt
# Les Salines
# Anse Dufour
# Grande Anse
```



Question flash

Après f.read(), que renvoie f.readline() ? Pourquoi ? Vérifiez dans la console.



Le \n qui traîne

readline() garde le saut de ligne. strip() (ou rstrip("\n")) le retire. Oublier ça = lignes vides mystérieuses à l'affichage.

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Huneil - 2026-2027

12

12

Écrire : write() et writelines()

- **f.write(ch)**
 - écrit la chaîne ch ; n'ajoute pas de \n tout seul ; renvoie le nombre de caractères écrits.
- **f.writelines(liste)**
 - écrit chaque chaîne de la liste, à la suite, sans séparateur.
- **print(..., file=f)**
 - le print habituel, redirigé vers le fichier — avec le \n automatique.
- *On ne peut écrire que des chaînes : write(42) est une erreur, write(str(42)) fonctionne.*

```
with open("sortie.txt", "w") as f:
    f.write("Relevé du jour\n")
    f.writelines(["28.5\n", "29.1\n"])
    print("fin", file=f)

with open("sortie.txt", "a") as f:
    f.write("ligne ajoutée plus tard\n")
```



Avec l'IA : le test qui fait mal

Demandez : « Que se passe-t-il si j'écris dans un fichier ouvert en mode "r" ? Et si le dossier n'existe pas ? » Puis provoquez ces erreurs vous-même et lisez le message d'exception. Vous saurez le reconnaître le jour J.

Exemple complet : recopier un fichier

Boucle tant que + readline

```
# Version « historique »
fic1 = open("source.txt", "r")
fic2 = open("but.txt", "w")
ch = fic1.readline()
while ch != "":
    fic2.write(ch)
    ch = fic1.readline()
fic1.close()
fic2.close()
```

with + for

```
# Version « pythonique »
with open("source.txt") as src, \
    open("but.txt", "w") as dst:
    for ligne in src:
        dst.write(ligne)
```



Pourquoi la boucle s'arrête ?

readline() renvoie "" à la fin du fichier, et seulement là (une ligne vide vaut "\n", pas ""). C'est déjà un raisonnement de terminaison : on y reviendra au chapitre Preuve.



Défi : le filtre

Recopier data.txt dans data_propre.txt en supprimant toutes les lignes qui commencent par #. Puis : et si on veut réécrire dans data.txt lui-même ? (Piège !)

PARTIE 3

Les fichiers CSV



Le format le plus simple du monde, et pourtant partout : Excel, banques, capteurs, Météo-France...

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Hunel - 2026-2027

15

15

CSV : Comma Separated Values

- Un fichier texte où chaque ligne est un enregistrement et chaque valeur est séparée par un séparateur : virgule, point-virgule, tabulation...
- La première ligne contient souvent les noms des colonnes (l'en-tête).
- Ouvert dans un tableur, il apparaît comme une grille. Ouvert dans un éditeur, c'est du texte.
- *Piège franco-français : Excel en français enregistre souvent avec « ; » car la virgule sert de séparateur décimal.*

etudiants.csv

```
commune,prenom,age
Le Lamentin,Jean,21
Fort-de-France,Claire,19
Sainte-Anne,Kévin,20
```

commune	prenom	age
Le Lamentin	Jean	21
Fort-de-France	Claire	19
Sainte-Anne	Kévin	20

Le même fichier vu par un tableur

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Hunel - 2026-2027

16

16

Lire et écrire un CSV avec le module csv

```
Lecture : csv.reader
import csv
with open("etudiants.csv", newline="") as f:
    lecteur = csv.reader(f, delimiter=",")
    entete = next(lecteur)      # lire ligne
    for ligne in lecteur:
        print(ligne)
# ['Le Lamentin', 'Jean', '21'] <- une liste
# attention : '21' est une chaîne !
```

```
Lecture par nom de colonne : DictReader
with open("etudiants.csv", newline="") as f:
    lecteur = csv.DictReader(f)
    for ligne in lecteur:
        print(ligne["prenom"], int(ligne["age"]))
```

```
Ecriture : csv.writer
with open("moyennes.csv", "w", newline="") as f:
    w = csv.writer(f, delimiter=";")
    w.writerow(["prenom", "moyenne"])
    w.writerow(["Claire", 14.5])
```



Pourquoi pas ligne.split(",") ?

Parce qu'une valeur peut contenir une virgule : "Fort-de-France, centre". Le module csv gère les guillemets, les séparateurs et les retours à la ligne. Ne réinventez pas la roue (surtout carrée).



Avec l'IA

Donnez-lui 3 lignes de votre CSV et demandez : « Génère-moi 50 lignes réalistes de test, avec quelques cas piégeux (virgules dans les valeurs, âge manquant). »

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Huneil - 2026-2027

17

17

Projet fil rouge : le carnet de relevés

Un mini-projet pour tout mettre ensemble — et qu'on retrouvera au chapitre Hachage sous une autre forme.

1

Saisir

Le programme demande une station (ex. « Trinité »), une température et une hauteur de vague, et les ajoute à relevés.csv (mode "a").

2

Consulter

Afficher tous les relevés d'une station donnée, triés par date.

3

Résumer

Calculer la moyenne par station et l'écrire dans resume.csv.



Contraintes

- Utiliser with pour chaque ouverture.
- Si relevés.csv n'existe pas, le créer avec son en-tête.
- Ne jamais planter sur une ligne mal formée : l'ignorer et compter les lignes ignorées.



Avec l'IA : partenaire, pas pilote

Autorisé : lui demander de générer un relevés.csv de test, de relire votre code, de vous expliquer une exception. Interdit : lui demander le projet entier. Vous devrez présenter votre code en 3 minutes, sans notes.

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Huneil - 2026-2027

18

18

Quiz flash



On ouvre `notes.txt` avec `open("notes.txt", "w")` puis on ne fait rien et on ferme. Que contient `notes.txt` ?

- A Ce qu'il contenait avant : on n'a rien écrit.
- B Rien : le mode "w" a vidé le fichier à l'ouverture.
- C Une ligne vide.
- D Erreur : on ne peut pas ouvrir sans écrire.

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Huneil - 2026-2027

19

19

Ce qu'il faut retenir

- `os` et `os.path` : se repérer et manipuler les chemins de façon portable (`join` !).
- `shutil` : copier, déplacer.
- `open(nom, mode)` → objet fichier ; "r", "w" (écrase !), "a". Toujours fermer : `with` le fait pour vous.
- Lecture : `read`, `readline` ("" à la fin), `readlines`, `for` ligne in `f`. Pensez au `\n`.
- Écriture : `write` (chaînes seulement), `writelines`, `print(file=f)`.
- CSV : module `csv`, `reader/DictReader/writer`, attention au délimiteur et aux types (tout est chaîne).



Réviser avec l'IA

« Fais-moi passer un oral de 5 questions sur les fichiers en Python. Pour chaque réponse, dis-moi ce qui manque, sans donner la réponse complète. »



Prochain épisode

Épisode 2 — Récursivité : des fonctions qui s'appellent elles-mêmes... et qui s'arrêtent (promis).

Licence Informatique 2e année - Algo & Prog LS3 - Ph. Huneil - 2026-2027

20

20